

第29-30课 PDF解析整理版

来源：飞书「ChatBI 项目系统蓝图」解析整理版。原课件 PDF 附件无下载权限，本文为文档正文导出。

第 29-30课 PDF 解析整理版

第29+30课：ChatBI 系统收口与项目复盘简历包装

[要点] 课程总结与关系梳理

这两节课共同构成了一个企业级AI项目从“技术实现”到“经验表达”的完整闭环。

– 第29课：系统收口与配置化重构

关注的是项目内部的工程化治理。它解决了系统功能增多后带来的配置分散、职责混杂、维护困难等问题。通过将系统重构为清晰的配置层、运行层和能力模块层，使项目从一个“能跑”的原型，转变为一个结构清晰、易于维护和扩展、更接近可交付状态的工程系统。

– 第30课：项目复盘与简历包装

关注的是如何将项目本身的价值清晰地对外传达。它指导你如何梳理项目脉络、提炼技术亮点，并最终形成能在1分钟内讲清楚的项目概述、有吸引力的简历条目以及应对面试高频问题的回答框架。

核心关系：第29课让你做出一个扎实、可交付的项目；第30课教你如何讲好这个项目。前者是“里子”，确保项目经得起技术推敲；后者是“面子”，确保你的工作和能力能被他人准确理解和认可。两者结合，才能将一个学习项目转化为有价值的、能体现你综合能力（技术+表达）的项目经验。

第 29 课：系统收口与配置化重构：让 ChatBI 更接近可交付项目

一、本节目标

1.1 课程定位

回顾下前面的两节课：

- 第 27 课把权限控制接进了查询主链路
- 第 28 课把执行阶段的异常分类、连接池和慢查询记录补齐了

因此，系统已经具备了完整查询、权限校验和执行治理能力。

当前问题不再是“能不能跑起来”，而是“代码、系统还能不能、好不好继续维护”。

功能越来越多以后，配置分散、开关分散、模块职责变重，会先成为新的问题。

1.2 当前问题

结合当前代码，能看到下面几类现象：

- code/chatbi_mvp/config.py 同时管理数据库连接、运行时参数和模型配置
- code/chatbi_mvp/main.py 直接组装解析、Prompt、模型、数据库、格式化等 多个模块
- code/chatbi_mvp/api_service.py 需要透传越来越多的运行开关
- code/chatbi_mvp/database.py 既关心数据库执行，也承接了一部分运行时约束

这个结构，未来一定会出现的长期问题：

- 配置项越来越多，入口越来越乱
- 改动一个开关，往往要同时改多个文件

1.3 本节目标

系统收口与配置化重构：

- 看清当前代码里的配置分散和职责混杂
- 把系统整理成 配置层、运行层、能力模块层 3 个边界
- 收拢功能开关、运行参数和环境配置
- 让主链路保留统一入口，同时减少直接依赖

二、现状分析与架构设计

2.1 当前代码状态

当前已经完成的结构：

这张图里最明显的问题——职责混在一起：

- main.py 既是查询入口，又在直接装配模块
- api_service.py 既在处理接口契约，也在直接接收大量运行开关
- config.py 同时给入口层、模型层、数据库层提供默认值

- 数据库、模型、能力模块都被主链路直接知道

会带来的长期问题：

- 主入口越来越重
- 配置改动越来越容易扩散

当前面临的问题：如何把这张图里的职责重新摆回更合适的位置？

2.2 真实症状案例

如果有如下一个具体的改动需求：

- 现状：把 `use_schema_linking` 从“请求里传什么就用什么”
- 需求目标：要求改成“系统有默认值，请求可以覆盖默认值”的逻辑

在当前版本的系统结构里，这个改动会同时牵动多个文件：

- `config.py` 里补默认值
- `api_service.py` 里接收请求参数
- `main.py` 里决定最终生效的开关
- 测试里把真实数据库、真实模型调用替换成可控的对象，再单独检查默认值逻辑

这种“小改动也会扩散”的问题，应该如何避免/解决？

2.3 目标结构

整理之后，更合适的结构如下：

- 配置层：回答默认行为和环境差异
- 运行层：负责模块装配和统一入口
- 能力模块层：处理具体能力

乍一看，好像同样还是这些模块，他们的职责边界到底发生了什么变化。

新的结构内，真正的区别在于：

- 配置不再散落读取
- 模块不再在主入口里直接装配

- API 层和主链路尽量少承担系统职责

2.4 前后调用链对照

先看重构前的调用链：

再看重构后的调用链：

优化后的调用链，重点在于：

- 配置读取位置更集中
- 模块装配不再塞在主入口里
- 请求覆盖和系统默认值先合并，再进入主链路
- 后续新增配置或能力时，改动路径更稳定

三、改造实现

3.1 改造范围

这一节先聚焦 4 个已有文件：

3.2 配置层

3.2.1 当前现状

当前 `config.py` 的特点：

- `DB_CONFIG` 管数据库连接
- `DB_RUNTIME_CONFIG` 管连接池和超时
- `LLM_CONFIG` 管模型调用

这一层可以作为基础版“统一配置入口”的实现。

3.2.2 配置项扩充

配置层负责控制“系统怎么启动、默认行为是什么”，回答下面几类问题：

- 当前环境用什么数据库连接

- 当前环境用什么模型
- 哪些功能默认打开
- 运行时阈值是多少

配置层的目标，是让“改参数”和“改业务逻辑”分开，配置项可以分成 3 组：

- 环境配置：数据库地址、账号、模型地址、密钥
- 运行配置：连接池大小、超时、慢查询阈值
- 功能开关：use_schema_linking、use_indicator_rag、use_rules

因此可以整理成下面这类结构：

```
APP_CONFIG = {"environment": "dev", "database": {"default_source": "mysql_main", "sources": {"mysql_main": {"driver": "mysql", "host": "localhost", "port": 3307, "database": "chatbi_mvp",
    }
    }, "runtime": {"pool_size": 5, "pool_timeout": 3, "slow_query_threshold_ms": 200,
    },
    }, "llm": {"model": "gpt-4", "embedding_model": "text-embedding-3-large",
    }, "features": {"schema_linking": True, "indicator_rag": True, "rules": True, "guards": True,
    },
    }
```

3.2.3 请求开关合并

当前 API 请求体里已经有多组开关：

- use_few_shot
- use_rules
- use_guards
- use_indicator_knowledge
- use_schema_linking
- use_indicator_rag

这组开关的潜在问题：

- 默认值放在哪里
- 哪些开关是环境默认
- 哪些开关允许请求侧覆盖（该开关用户是否可以切换？）

因此我们至少需要将其整理成两层，并添加合并逻辑：

1. 系统默认配置
2. 请求级覆盖参数

```
def _resolve_query_options(payload: QueryRequest, app_config: dict) -> dict[str, bool]:
    feature_defaults = app_config.get("features", {})
    return {"use_few_shot": payload.use_few_shot if payload.use_few_shot is
not None else feature_defaults.get("few_shot", False), "use_rules": payload.use_rules if payload.use_rules is not None else
feature_defaults.get("rules", False), "use_guards": payload.use_guards if payload.use_guards is not None else
feature_defaults.get("guards", False), "use_indicator_knowledge": (
        payload.use_indicator_knowledge if payload.use_indicator_knowledge is not None else
feature_defaults.get("indicator_knowledge", False)
    ), "use_schema_linking": (
        payload.use_schema_linking if payload.use_schema_linking is not None else feature_defaults.get("schema_linking",
False)
    ), "use_indicator_rag": (
        payload.use_indicator_rag if payload.use_indicator_rag is not None else feature_defaults.get("indicator_rag", False)
    ),
    }
```

目的：让接口层、主链路和配置层的关系更清楚。

3.3 运行层

在重构前，main.py 的初始化步骤中直接创建了这些对象：

- QueryParser
- LLMClient
- DatabaseClient
- ResultFormatter
- IndicatorKnowledge

主入口既在“跑查询”，也在“装系统”：

```
class ChatBISystem: """ChatBI 系统主类"""
    def init(self):
        self.parser = QueryParser()
        self.llm = LLMClient()
        self.db = DatabaseClient()
        self.formatter = ResultFormatter()
        self.indicator_knowledge = IndicatorKnowledge()
```

整理之后，可以收成下面这种形式：

```
class ChatBISystem: """ChatBI 系统主类"""
    def init(
        self,
        runtime=None,
        app_config: dict | None = None,
        runtime_factory=build_runtime,
    ):
```

```

self.app_config = app_config or APP_CONFIG
self.runtime_factory = runtime_factory
self.runtime = runtime or self.runtime_factory(self.app_config)
self.parser = self.runtime.parser
self.llm = self.runtime.llm
self.db = self.runtime.db
self.formatter = self.runtime.formatter
self.indicator_knowledge = self.runtime.indicator_knowledge

```

真正的实例化动作放到单独的工厂里：

```

def build_runtime(app_config: dict | None = None, source_id: str | None = None) -> AppRuntime:
    config = app_config or APP_CONFIG
    resolved_source_id = source_id or config["database"]["default_source"]
    return AppRuntime(
        source_id=resolved_source_id,
        parser=QueryParser(),
        llm=LLMClient(),
        db=build_database_client(config, resolved_source_id),
        formatter=ResultFormatter(),
        indicator_knowledge=IndicatorKnowledge(),
    )

```

这样改完以后，main.py 更像统一入口，而装配细节则被收进运行层，获取 runtime 的方法（单例可复用、支持多 runtime）如下：

```

def _get_runtime(self, source_id: str | None = None):
    if source_id is None or source_id == self.runtime.source_id:
        return SimpleNamespace(
            source_id=self.runtime.source_id,
            parser=self.parser,
            llm=self.llm,
            db=self.db,
            formatter=self.formatter,
            indicator_knowledge=self.indicator_knowledge,
        )
    return self.runtime_factory(self.app_config, source_id=source_id)

```

四、案例与验证

4.1 多开关散落案例

回到前面具体的一个改动需求：

- 现状：把 use_schema_linking 从“请求里传什么就用什么”
- 需求目标：要求改成“系统有默认值，请求可以覆盖默认值”的逻辑

改造前，这类改动容易散到：

- config.py
- api_service.py

- main.py
- 测试代码

改造后，这类改动主要收敛到两处：

- 配置层定义默认值 (config.py)
- API 层统一合并请求覆盖参数 (api_services._resolve_query_options, 对应 3.2.3 小节)

这样主入口不需要再额外承担默认值判断。

4.2 扩展性案例：接入多数据源

如果当前 ChatBI 系统未来需要考虑接入多数据源的需求（如面向不同环境、不同应用 场景）

第一步：把“数据源”也纳入配置层

```
"database": {"default_source": "mysql_main", "sources": {"mysql_main": {"driver": "mysql", "host": "localhost", "port": 3307, "database": "chatbi_mvp",
    }, "finance_pg": {"driver": "postgresql", "host": "localhost", "port": 5432, "database": "finance_dw",
    },
    },
}
```

第二步：给运行层补一个选择逻辑

```
source_id = source_id or app_config["database"]["default_source"]
db = build_database_client(app_config["database"]["sources"][source_id])
```

实现效果：如果以后数据库类型变了（或增加新类型），改动主要落在配置层和装配层，主链路不需要跟着全面改写。

- APP_CONFIG["database"]["sources"]
- build_database_client()
- 运行时选择逻辑

4.3 单元测试

验证 1：运行时工厂是否能按指定 source_id 装配数据库实例

- 不走真实数据库连接，而是先把 runtime_factory.py 里会用到的真实对象临时替换成假对象
- 验证“装配逻辑是否正确”

```
uv run pytest tests/test_runtime_factory.py::test_build_runtime_uses_requested_database_source -q
```

验证 2: ChatBISystem.run() 是否能正确透传 source_id

```
uv run pytest tests/test_runtime_factory.py::test_chatbi_system_run_uses_runtime_factory_for_source_id -q
```

验证 3: 请求级参数是否会和系统默认配置正确合并

```
uv run pytest tests/test_api_query_options.py -q
```

五、附录

本节改动涉及的相关代码:

- code/chatbi_mvp/config.py
- code/chatbi_mvp/main.py
- code/chatbi_mvp/api_service.py
- code/chatbi_mvp/database.py
- code/chatbi_mvp/runtime_factory.py

第 30 课: 项目复盘与简历包装: 如何讲清你的企业 AI 项目

一、本节目标

1.1 课程定位

前面的 29 课, 已经把企业级 ChatBI 系统从业务问题一步步做到了可运行、可分析、可治理、可收口。

现在还差最后一步: 把这个项目整理成你能稳定讲清楚的项目经验:

- 一段 1 分钟内能讲清楚的项目总述
- 3 到 5 条可写进简历的项目亮点
- 4 到 6 个可继续展开的扩展方向
- 一套高频追问的回答顺序

1.2 当前问题

很多人学习项目时的误区, 表达会卡在下面几种情况:

- 只会说用了 LLM、RAG、Agent

- 能讲局部实现，但讲不清完整链路、上下游依赖（哪怕不是你自己做的）
- 简历里堆技术词，别人看不出你到底解决了什么问题
- 面试追问一深，就分不清哪些是已经做过的，哪些只是想法（不知道业界现在最牛的能做到什么程度）

项目表达里最重要的，一定不是专业术语多，而是让别人快速听懂：

- 业务场景是什么（S, Situation）
- 系统任务有哪些，为什么难（T, Task）
- 你做了哪些关键设计（A, Action）
- 结果如何，怎么验证（R, Result）

二、项目内容回顾

2.1 六阶段项目复盘

先把整套项目重新收成一条主线：

这条主线里，每一阶段都回答了一个具体的问题：

项目复盘时的合理顺序：业务问题 -> 系统难点 -> 分阶段解决 -> 最终形成什么能力。

2.2 项目 4 大亮点

亮点 1：真实的业务问题

- 场景不是泛化问答，而是瞄准了真实制造业的财务分析流程
- 问题不仅是“Text2SQL 查一个数”，还包括“利润为什么下降”等复杂、真实问题
- 系统必须同时处理查询、数据理解和分析推理

亮点 2：完整的系统演进链路

- 从最小 Text2SQL 开始
- 再补服务化、交互、检索、Agent 和工程治理
- 让别人（面试官）感受到这是一个你长时间参与、逐步成长起来的系统，不是技术拼凑+名次堆叠

亮点 3：有代表性的技术难点

- Text2SQL 容易字段选错
- 企业 Schema 大了之后，全量注入必定不可持续
- 指标定义有口径和依赖链
- 复杂分析天然需要多步计划和中间结果管理

亮点 4：完整的工程化思考

- 对 API 层和流式输出的设计
- 权限控制、安全拦截、执行异常分类
- 慢查询记录、连接池、配置化重构与未来扩展性思考

2.3 简历里写什么

如果简历篇幅有限，针对该项目建议优先写下面这些内容：

表格

简历里的每一条内容，都尽量回答 3 个问题：

- 你做了什么
- 为什么要这样做
- 做完之后系统有什么变化

三、项目表达

3.1 项目口述

可参考的稳定表达结构：

1. 用一句话交代业务场景
2. 用一句话交代核心难点
3. 用两到三句话说明系统主链路
4. 用一句话补工程化和结果

可参考口述版本：

我参与主导了一个面向制造业财务分析场景的企业级
人员用自然语言完成复杂数据分析。

ChatBI

系统，目标是让业务

这个项目的难点，不只是把问题转成 SQL，还要解决企业数据里的表字段歧义、指标口径理解、多步分析拆解，以及真实服务环境下的权限和性能问题。

我把系统拆成了 Text2SQL 主链路、Schema Linking 与指标 RAG、Plan-and-Execute Agent，以及服务化治理 4 个部分，逐步补齐了查询理解、数据理解、复杂分析和工程交付能力。

最后系统支持从自然语言提问到 SQL 生成、数据库执行、结果总结的完整闭环，并补齐了流式接口、权限控制、执行异常分型、慢查询记录和配置化重构。

3.2 简历条目撰写

一种信息量偏低的写法：

- 基于 OpenAI、FastAPI、LangChain 搭建了一个 ChatBI 系统
- 实现了 Text2SQL、RAG、Agent
- 完成了前后端联调和项目优化

这几句的问题：

- 技术词很多，但业务目标不清楚
- 看不出系统难点
- 看不出你真正负责了什么

建议改写形式：

- 面向制造业财务分析场景，设计并实现企业级 ChatBI 系统，围绕“利润下降原因分析”等开放式问题，完成自然语言查询到数据分析结果的主链路搭建
- 针对企业数据表多、字段歧义强、指标口径复杂的问题，落地 Schema Linking 与指标 RAG 方案，提升相关表字段和指标定义的动态召回能力
- 设计 Plan-and-Execute Agent 流程，将复杂分析问题拆为多步 SQL 任务，并补齐中间结果管理与分析报告生成
- 完成 FastAPI 服务化、SSE 流式接口、权限控制、SQL 执行异常分型、慢查询记录与配置化重构

3.3 面试回答框架

面试时的高频追问：

- 为什么这个项目不能只做单条 SQL
- 为什么需要 Schema Linking
- 指标 RAG 和普通字段检索有什么区别
- Agent 方案和直接生成 SQL 的差别是什么
- 这个项目里最难的工程问题是什么

面对任何面试官的追问问题，均可以采用如下标准的回答框架：

1. 先说问题表现
2. 再说根因
3. 再说方案
4. 最后说验证和边界

例：“为什么需要 Schema Linking？”

示例回答：直接全量注入 Schema，在小表、少表场景下能工作，但我们的场景里表 and 字段一多，问题会很明显：一是 Token 消耗高，二是噪声多，三是相似字段会互相干扰。

所以这里要先做相关表召回，再做字段匹配，最后补 Join 路径推理，把真正和当前问题有关的 Schema 片段送进 Prompt。

这样做之后，系统不是单纯“知道的更多”，而是“给模型的上下文更准”，这也是后面指标 RAG 和执行链路能稳定工作的前提。

3.4 简历里怎么埋“钩子”

目标：你在简历或项目介绍里，主动露出一个你准备最充分、最容易讲深的点，让面试官顺着这个点追问，而不是把对话带到你不熟的地方。

埋钩子时，优先选下面这类内容：

- 你亲自参与最多的模块
- 你能讲清问题、根因、方案、验证的模块
- 既有已完成事实，又自然带出扩展空间的模块

不建议埋的钩子：

- 你只知道结论、不清楚实现细节的模块

- 你只会复述名词、答不出取舍逻辑的模块
- 明显还没做、但容易被追着问实现细节的方向

以当前项目为例，比较适合埋钩子的点有 4 类：

例如，你想把面试官往 Schema Linking 上引，可以这样写简历：

- 针对企业数据表多、字段歧义强的问题，设计并实现 Schema Linking 链路，完成表召回、字段匹配与 Join 路径推理，提升动态 Schema 上下文构造能力

这句话容易引出 3 类追问：

- 为什么不能直接把全量 Schema 放进 Prompt
- 表召回、字段匹配、Join 推理各自解决什么问题
- 如果继续优化，你会先做什么

好的面试钩子，应该把追问引到你准备最充分的地方。

它的目标不是“显得做了很多”，而是“让面试官顺着你最强的点继续问下去”。

3.5 高频面试题

3.5.1 为什么这个项目不能只做单条 SQL

课程章节：2、21

答题思路

- 先说单条 SQL 适合的边界
- 再说复杂分析任务为什么超出这个边界
- 再说明 Agent 的必要性

示范回答

单条 SQL 适合查一个明确指标，或者做一次简单聚合。但这个项目面对的是“利润为什么下降”这类分析型问题，它通常要先看趋势，再拆产品线、区域、费用项，还要把多步结果整理成统一结论。把这些都压进一条 SQL，SQL 会很长，也很难维护，所以后面我引入了 Plan-and-Execute Agent，把复杂问题拆成多步任务执行。

3.5.2 为什么需要 Schema Linking，而不是直接把全量 Schema 放进 Prompt

课程章节：13-20

答题思路

- 先说全量注入在小场景下可用
- 再说明企业场景里的 3 个问题
- 最后说 Schema Linking 具体补了哪几层能力

示范回答

全量 Schema 注入在表少、字段少的时候可以工作，但企业场景里很快会遇到 3 个问题：Token 成本高、无关字段太多、相似字段容易互相干扰。Schema Linking 的作用，就是先把和当前问题最相关的表、字段、Join 路径筛出来，再把精简后的上下文交给模型。这样模型不是知道得更多，而是拿到的信息更准。

3.5.3 指标 RAG 和普通字段检索有什么区别

课程章节：15、16、19

答题思路

- 先区分“字段”和“指标”不是同一层
- 再说指标为什么更难
- 最后说明 RAG 检索补的是业务定义

示范回答

字段检索解决的是“该用哪个字段”，比如 gross_amount 和 net_amount 选哪个。指标 RAG 解决的是“这个业务指标到底怎么算”，因为像利润、毛利率、销售费用这类指标，背后不仅有字段，还有口径说明、公式、过滤条件和依赖关系。所以我把指标定义做成知识库，用检索把相关定义动态注入给模型，避免模型只看字段名做猜测。

3.5.4 这个项目里，Text2SQL 最容易出错的地方是什么

课程章节：2、7、8

答题思路

- 先列 2 到 3 类典型错误

- 再说这些错误为什么在企业场景更明显
- 最后说明你是怎么逐步治理的

示范回答

最容易出错的地方主要有 3 类：字段选错、Join 写错、过滤条件遗漏。企业场景里这些问题更明显，因为字段语义相近、表关系更复杂，而且很多业务口径不是字段名能直接表达出来的。我前面先通过 Prompt 规则和错误分析做第一轮修复，后面再用 Schema Linking、指标知识和评估体系逐步把这些错误收窄。

3.5.5 你是怎么验证系统效果的

课程章节：7、8

答题思路

- 先说不能只看模型“感觉对不对”，而是需要一套测评体系
- 再说你用了哪些验证方法
- 最后说验证的意义

示范回答

这类系统不能只靠主观判断，所以我专门补了一套评估思路。前面会用真实问题构造 测试集，对 SQL 结果做执行级验证，观察生成 SQL 能不能真正查出正确结果。后面 在链路升级时，还会对比全量 Schema 和精简 Schema 的效果差异，观察准确性、上下文长度和响应表现的变化。这样每次优化就有依据、有方向，不是只看模型回答像不像。

3.5.6 链路做成服务化需要考虑哪些点

课程章节：10、11、12

答题思路

- 先说脚本版本适合验证思路
- 再说真实使用场景需要什么
- 最后说明服务化带来的能力

示范回答

脚本版本适合验证最小链路，能尽快看出自然语言到 SQL 这条路能不能跑通。但如果系统真的要被前端调用、被多人使用，就必须做服务化。服务化之后，接口契约、错误返回、流式输出、健康检查、权限上下文这些能力才能接进来，也更接近真实企业系统的使用方式。

3.5.7 SSE 流式输出的价值是什么

课程章节：11、12

答题思路

- 先说不用流式时的体验问题
- 再说流式解决了什么
- 最后补一句边界

示范回答

不用流式的时候，用户在等待 SQL 生成和执行结果的几秒里几乎没有反馈，体验上会觉得系统卡住了。SSE 流式输出的价值，是把生成过程分阶段返回，比如 SQL 逐步生成、执行状态更新、最终结果返回，用户能看到系统正在工作。它主要解决的是交互体验问题，不直接提升 SQL 准确率，但会明显改善产品可用性。

3.5.8 Agent 方案和直接生成长 SQL 相比，优势在哪里

课程章节：21、22、23、24、25、26

答题思路

- 先说长 SQL 的局限
- 再说 Agent 的拆解价值
- 最后说明 Agent 也有成本

示范回答

直接生成长 SQL 的问题是，一旦任务里同时包含趋势、归因、维度拆分和总结，SQL 会变得很长，而且任何一步出错都很难定位。Agent 的优势是把复杂问题拆成多个可执行步骤，每一步都能单独看输入、输出和中间结果，排错和扩展都更方便。当然 Agent 也会增加链路长度和调度复杂度，所以它更适合复杂分析任务，不是所有问题都要走 Agent（可以先做问题分类）。

3.5.9 你觉得这个项目里最有代表性的工程化能力是什么

课程章节：27、28、29

答题思路

- 先选 2 到 3 个最能体现工程化的点
- 再说明它们为什么重要
- 最后和 Demo 项目做区分

示范回答

我觉得最有代表性的工程化能力有 3 个：权限与安全治理、执行异常分型与慢查询记录、配置化重构。因为我在做这个 LLM 类的项目过程中发现，早期验证时往往只关注“能不能跑”，但要上线评审企业系统更关心“谁能查、出错怎么处理、后续怎么维护”，这几个点补齐之后，项目才能更接近真实交付状态。

3.5.10 如果继续扩展这个项目，你会优先做什么

课程章节：20、27、28、29

答题思路

- 先说明当前系统已经完成到哪一步
- 再给出 2 到 3 个下一步方向
- 最后说明优先级依据

示范回答

如果继续扩展，我会优先做 3 个方向。第一是多数据源接入（横向扩展多个业务部门），把当前单库查询扩展成统一元数据层和路由层；第二是检索优化，比如重排序、语义缓存和更细的召回策略；第三是企业治理能力，比如审计日志、权限中心和可观测性看板。【重点】表达优先级的判断标准：先补最影响真实交付的部分，再补 效果优化。

3.5.11 这个项目里，你个人贡献最值得强调的部分是什么

课程章节：18、19、20

答题思路

- 先选自己最熟的一段
- 再说这段为什么关键
- 最后补你在其中的具体动作

示范回答

如果让我选一个最值得强调的个人贡献，我会重点讲 Schema Linking 和主链路整合。因为这部分直接决定系统在企业复杂数据场景下能不能稳定工作，我做的不只是一个检索模块，而是把表召回、字段匹配和 Join 推理接进了 Prompt 构造链路里，并和后面的指标知识、执行链路一起打通，让系统从“能生成 SQL”走到“更有机会生成正确 SQL”。

3.5.12 这个项目最大的不足是什么

课程章节：29、30

答题思路

- 先承认真实边界
- 再说明为什么会有这个边界
- 最后给出清楚的改进方向

示范回答

这个项目目前最大的边界，是它已经覆盖了完整主链路，但还没有完全走到企业大规模生产环境。比如多数据源治理、更细的权限中心、统一审计、完整压测和可观测性 体系，还可以继续补。

注意：我们的处理方式不是回避这个问题，而是把“已完成的能力”和“下一步最优先补的能力”分开讲清楚，这样面试官会更容易判断项目的真实成熟度。

四、小结

4.1 本节收获

项目表达的目标，是让别人相信你真的做过、真的理解、真的能继续往下做。

- 先讲业务场景
- 再讲系统难点
- 再讲关键设计

- 最后讲验证结果和扩展边界

4.2 完整复盘清单

课后可以按这个清单再过一遍自己的项目表达：

- 我能不能用 1 分钟讲完整个项目
- 我能不能清楚说明为什么要做 Schema Linking
- 我能不能讲清 Agent 适合解决什么问题
- 我能不能区分系统能力和工程治理能力
- 我能不能明确说出哪些是已完成，哪些是下一步扩展

4.3 课后练习

练习 1：把项目压缩成 3 句话

要求：

- 第 1 句讲业务场景
- 第 2 句讲技术难点
- 第 3 句讲你的关键贡献

练习 2：从下面 3 个角度里，任选 1 个继续展开成简历条目

- Schema Linking
- Plan-and-Execute Agent
- 权限、安全与性能治理

可直接套用这个结构：

在 _____ 场景下，针对 _____ 问题，设计并实现了 _____ 方案，补齐了 _____ 能力。

附件（飞书无下载权限）：第 29 课：系统收口与配置化重构：让 ChatBI 更接近可交付项目.pdf

附件（飞书无下载权限）：第 30 课：项目复盘与简历包装：如何讲清你的企业 AI 项目.pdf